

4.13.61

Rushil Shanmukha Srinivas
EE25BTECH11057
Electrical Engineering ,
IIT Hyderabad.

September 30, 2025

1 Problem

2 Solution

- Dot Product and Locus
- Plots

3 C Code

4 Python Code

Problem Statement

Question : A rectangle $PQRS$ has its side PQ parallel to the line $y = mx$ and vertices P, Q and S on the lines $y = a, x = b$ and $x = -b$, respectively. Find the locus of the vertex R .

Dot Product and Locus

Solution:

Given, **P**, **Q** and **S** lie on the lines $y = a$, $x = b$ and $x = -b$.

$$\mathbf{n}_1^\top \mathbf{P} = a, \mathbf{n}_1 = \begin{pmatrix} 0 \\ 1 \end{pmatrix} \quad (3.1)$$

$$\mathbf{n}_2^\top \mathbf{Q} = b, \mathbf{n}_2^\top \mathbf{S} = -b, \mathbf{n}_2 = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \quad (3.2)$$

$$\mathbf{m} = \begin{pmatrix} 1 \\ m \end{pmatrix}, \mathbf{n} = \begin{pmatrix} -m \\ 1 \end{pmatrix}, \mathbf{m} \perp \mathbf{n} \quad (3.3)$$

As **PQ** is parallel to **m** and **PS** is parallel to **n**

$$\mathbf{Q} - \mathbf{P} = \mathbf{R} - \mathbf{S} = k_1 \mathbf{m}, \mathbf{S} - \mathbf{P} = k_2 \mathbf{n} \quad (3.4)$$

$$\mathbf{n}_2^\top \mathbf{m} = 1, \mathbf{n}_2^\top \mathbf{n} = -m \quad (3.5)$$

$$\mathbf{n}_1^\top \mathbf{m} = m, \mathbf{n}_1^\top \mathbf{n} = 1 \quad (3.6)$$

$$\mathbf{Q} = \mathbf{P} + k_1 \mathbf{m} \implies \mathbf{n}_2^\top \mathbf{Q} = \mathbf{n}_2^\top \mathbf{P} + k_1 \mathbf{n}_2^\top \mathbf{m} = b \quad (3.7)$$

$$\mathbf{S} = \mathbf{P} + k_2 \mathbf{n} \implies \mathbf{n}_2^\top \mathbf{S} = \mathbf{n}_2^\top \mathbf{P} + k_2 \mathbf{n}_2^\top \mathbf{n} = -b \quad (3.8)$$

$$k_1 + mk_2 = 2b \quad (3.9)$$

$$\mathbf{R} = \mathbf{P} + k_2 \mathbf{n} + k_1 \mathbf{m} \implies \mathbf{n}_1^\top \mathbf{R} = \mathbf{n}_1^\top \mathbf{P} + k_2 \mathbf{n}_1^\top \mathbf{n} + k_1 \mathbf{n}_1^\top \mathbf{m} \quad (3.10)$$

$$\mathbf{n}_1^\top \mathbf{R} = a + k_2 + k_1 m \quad (3.11)$$

$$\mathbf{R} = \mathbf{P} + k_2 \mathbf{n} + k_1 \mathbf{m} \implies \mathbf{n}_2^\top \mathbf{R} = \mathbf{n}_2^\top \mathbf{P} + k_2 \mathbf{n}_2^\top \mathbf{n} + k_1 \mathbf{n}_2^\top \mathbf{m} \quad (3.12)$$

$$\mathbf{n}_2^\top \mathbf{P} + k_1 \mathbf{n}_2^\top \mathbf{m} = b \quad (3.13)$$

$$\mathbf{n}_2^\top \mathbf{R} = b - mk_2 \implies k_2 = \frac{b - \mathbf{n}_2^\top \mathbf{R}}{m} \quad (3.14)$$

$$k_1 + mk_2 = 2b \implies k_1 + b - \mathbf{n}_2^\top \mathbf{R} = 2b \quad (3.15)$$

$$k_1 = b + \mathbf{n}_2^\top \mathbf{R} \quad (3.16)$$

$$\mathbf{n}_1^\top \mathbf{R} = a + k_2 + k_1 m \implies \mathbf{n}_1^\top \mathbf{R} = a + \frac{b - \mathbf{n}_2^\top \mathbf{R}}{m} + mb + m\mathbf{n}_2^\top \mathbf{R} \quad (3.17)$$

$$\mathbf{n}_1^\top \mathbf{R} = a + b \left(m + \frac{1}{m} \right) + \mathbf{n}_2^\top \mathbf{R} \left(m - \frac{1}{m} \right) \quad (3.18)$$

$$\mathbf{n}_2^\top \mathbf{R} \left(\frac{1}{m} - m \right) + \mathbf{n}_1^\top \mathbf{R} = a + b \left(m + \frac{1}{m} \right) \quad (3.19)$$

$$\begin{pmatrix} \frac{1}{m} - m & 1 \end{pmatrix} \begin{pmatrix} \mathbf{n}_2^\top \mathbf{R} \\ \mathbf{n}_1^\top \mathbf{R} \end{pmatrix} = a + b \left(m + \frac{1}{m} \right) \quad (3.20)$$

$$\mathbf{R} = \begin{pmatrix} \mathbf{n}_2^\top \mathbf{R} \\ \mathbf{n}_1^\top \mathbf{R} \end{pmatrix} \quad (3.21)$$

Let

$$c = a + b \left(m + \frac{1}{m} \right) \text{ and } \mathbf{n}_3 = \begin{pmatrix} \frac{1}{m} & -m \\ & 1 \end{pmatrix} \quad (3.22)$$

The locus of the vertex $\mathbf{R}$ is

$$\mathbf{n}_3^\top \mathbf{R} = c \quad (3.23)$$

Plots

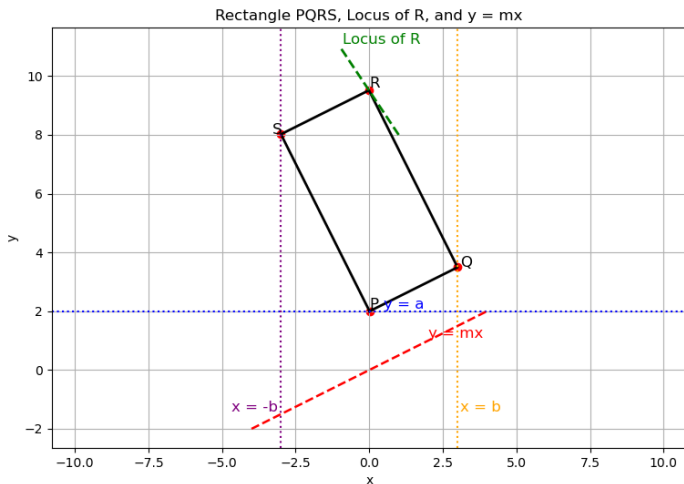


Figure: fig : Representation of Rectangle and Locus

C Code

```
#include <stdio.h>

/*
A rectangle PQRS has its side PQ parallel to the line  $y = mx$ 
and vertices P, Q, S on lines  $y=a$ ,  $x=b$ ,  $x=-b$  respectively.
Find the locus of vertex R.
Result:  $y = m*x - (a*(1+m^2))/m$ , with  $x = b$ 
*/

double find_locus(double m, double a, double x) {
    return m * x - (a * (1 + m * m)) / m;
}

int main() {
    double m, a, b;

    printf("Enter slope-m:-");
    scanf("%lf", &m);
```

```
printf("Enter constant a (line y=a): ");  
scanf("%lf", &a);
```

```
printf("Enter constant b (lines x=b): ");  
scanf("%lf", &b);
```

```
// For  $x = +b$   
double y1 = find_locus(m, a, b);  
printf("For  $x = %.2f$ ,  $R = (%.2f, %.2f) \setminus n$ ", b, b, y1);
```

```
// For  $x = -b$   
double y2 = find_locus(m, a, -b);  
printf("For  $x = %.2f$ ,  $R = (%.2f, %.2f) \setminus n$ ", -b, -b, y2);
```

```
return 0;
```

```
}
```

Python : call_c.py

```
import ctypes

# Load the compiled shared object
# Make sure rectangle_locus.so is in the same directory
lib = ctypes.CDLL('./rectangle_locus.so')
# Define the argument and return types of the C function
# double find_locus(double m, double a, double x)
lib.find_locus.argtypes = [ctypes.c_double, ctypes.c_double, ctypes.c_double]
lib.find_locus.restype = ctypes.c_double

def call_find_locus(m, a, b):
    results = [ ]
    for x in [b, -b]:
        y = lib.find_locus(m, a, x)
        results.append((x, y))
    return results
```

```
if __name__ == "__main__":  
    # Example values (you can change these)  
    m, a, b = 2.0, 3.0, 4.0  
  
    points = call_find_locus(m, a, b)  
    for x, y in points:  
        print(f'R = ({x:.2f}, {y:.2f})')
```

Python Code for Plotting

```
import numpy as np
import matplotlib.pyplot as plt

# Parameters
m = 0.5 # slope of PQ and the reference line  $y = mx$ 
a = 2 #  $y = a$ 
b = 3 #  $x = b$ 
x_P = np.linspace(-1, 1, 100) # vary x-coordinate of P

# Coordinates of Q, S, R
y_Q = m*(b - x_P) + a
y_S = a + (b + x_P)/m
x_R = -x_P
y_R = y_Q + y_S - a

# Pick an example rectangle
```

```
example_idx = 50
P = (x_P[example_idx], a)
Q = (b, y_Q[example_idx])
S = (-b, y_S[example_idx])
R = (x_R[example_idx], y_R[example_idx])

# Rectangle vertices order for plotting
rect_x = [P[0], Q[0], R[0], S[0], P[0]]
rect_y = [P[1], Q[1], R[1], S[1], P[1]]
plt.figure(figsize=(9,6))
# Plot rectangle
plt.plot(rect_x, rect_y, 'k-', linewidth=2)
plt.scatter([P[0], Q[0], R[0], S[0]], [P[1], Q[1], R[1], S[1]], color='red')

# Plot locus of R
plt.plot(x_R, y_R, 'g--', linewidth=2)
plt.text(x_R[-1]+0.1, y_R[-1], 'Locus of R', color='green', fontsize=12,
         verticalalignment='bottom')
```

```
# Constraint lines and labels
plt.axhline(a, color='blue', linestyle=':', linewidth=1.5)
plt.text(0.5, a+0.1, 'y = a', color='blue', fontsize=12)

plt.axvline(b, color='orange', linestyle=':', linewidth=1.5)
plt.text(b+0.1, -1.5, 'x = b', color='orange', fontsize=12,
         verticalalignment='bottom')

plt.axvline(-b, color='purple', linestyle=':', linewidth=1.5)
plt.text(-b-0.1, -1.5, 'x = -b', color='purple', fontsize=12,
         horizontalalignment='right', verticalalignment='bottom')

# Plot  $y = mx$  line passing through origin
x_line = np.linspace(-4, 4, 100)
y_line = m * x_line
plt.plot(x_line, y_line, 'r--', linewidth=1.8)
plt.text(2, m*2+0.1, 'y = mx', color='red', fontsize=12)
```

```
# Label rectangle vertices
plt.text(P[0], P[1]+0.1, 'P', fontsize=12)
plt.text(Q[0]+0.1, Q[1], 'Q', fontsize=12)
plt.text(R[0], R[1]+0.1, 'R', fontsize=12)
plt.text(S[0]-0.3, S[1], 'S', fontsize=12)

# Set limits to see full rectangle and lines
plt.xlim(-4, 4)
plt.ylim(min(min(y_R), -2), max(max(y_R), 4))

plt.xlabel('x')
plt.ylabel('y')
plt.title('Rectangle PQRS, Locus of R, and  $y = mx$ ')
plt.grid(True)
plt.axis('equal')
plt.savefig("../figs/locus.png")
plt.show()
```